NAME : NITHIN.V

ROLL.NO : 2403A52355

BATCH.NO : 13(AIML)

STEP 1 — Import Libraries

```
# Basic libraries
import numpy as np                    # For numerical computations
import pandas as pd                   # For dataset handling

# Text preprocessing
import re                             # For text cleaning using regex
from sklearn.model_selection import train_test_split  # For train-test split
from sklearn.metrics import classification_report, confusion_matrix

# Deep Learning
import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, Conv1D, GlobalMaxPooling1D, Dense, Dropout

# Visualization
import matplotlib.pyplot as plt
import seaborn as sns
```

STEP 2 — Load and Explore Dataset

```
# Load SMS Spam dataset (tab separated)
data = pd.read_csv("SMSSpamCollection",
                   sep='\t',
                   header=None,
                   names=['label', 'message'])



# Display first 5 rows
print(data.head())

# Check dataset shape
print("Dataset shape:", data.shape)

# Check class distribution
print(data['label'].value_counts())
```

```
  label                                            message
0   ham  Go until jurong point, crazy.. Available only ...
1   ham                      Ok lar... Joking wif u oni...
2  spam  Free entry in 2 a wkly comp to win FA Cup fina...
3   ham  U dun say so early hor... U c already then say...
4   ham  Nah I don't think he goes to usf, he lives aro...
Dataset shape: (5572, 2)
label
ham     4825
spam     747
Name: count, dtype: int64
```

STEP 3 — Text Preprocessing

```
# Convert labels to binary
data['label'] = data['label'].map({'ham':0, 'spam':1})

# Text cleaning function
def clean_text(text):
    text = text.lower()                           # Convert to lowercase
    text = re.sub(r'[^a-zA-Z\s]', '', text)       # Remove punctuation & numbers
    return text

# Apply cleaning
data['message'] = data['message'].apply(clean_text)

# Tokenization
tokenizer = Tokenizer(num_words=10000)            # Keep top 10k words
```

```
tokenizer.fit_on_texts(data['message'])

sequences = tokenizer.texts_to_sequences(data['message'])

# Padding sequences
max_length = 100
X = pad_sequences(sequences, maxlen=max_length)

y = data['label'].values
```

STEP 4 — Vocabulary and Embedding Preparation

```
# Vocabulary size
word_index = tokenizer.word_index
vocab_size = len(word_index) + 1
print("Vocabulary size:", vocab_size)

# Download GloVe manually and unzip glove.6B.100d.txt
embedding_dim = 100

# Load GloVe embeddings
embeddings_index = {}
with open("glove.6B.100d.txt", encoding="utf8") as f:
    for line in f:
        values = line.split()
        word = values[0]
        vector = np.asarray(values[1:], dtype='float32')
        embeddings_index[word] = vector

print("Loaded GloVe vectors:", len(embeddings_index))

# Create embedding matrix
embedding_matrix = np.zeros((vocab_size, embedding_dim))

for word, i in word_index.items():
    if i < vocab_size:
        embedding_vector = embeddings_index.get(word)
        if embedding_vector is not None:
            embedding_matrix[i] = embedding_vector
```

```
Vocabulary size: 8630
Loaded GloVe vectors: 259556
```

STEP 5 — Train–Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

print("Training size:", X_train.shape)
print("Testing size:", X_test.shape)
```

```
Training size: (4457, 100)
Testing size: (1115, 100)
```

STEP 6 — Build 1D CNN Model with Pretrained Embeddings

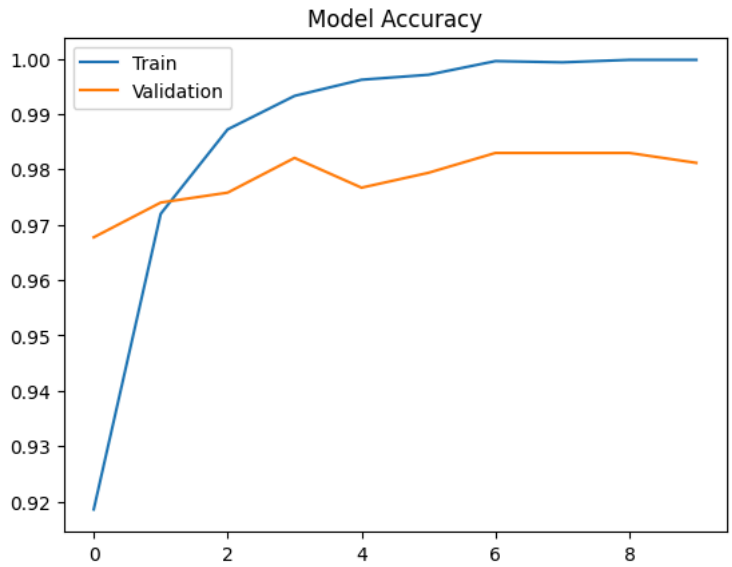
```
model = Sequential()

# Embedding layer using pretrained weights
model.add(Embedding(input_dim=vocab_size,
                    output_dim=embedding_dim,
                    weights=[embedding_matrix],
                    input_length=max_length,
                    trainable=False))   # Freeze embeddings

# 1D CNN layer
model.add(Conv1D(filters=128, kernel_size=5, activation='relu'))

# Global Max Pooling
model.add(GlobalMaxPooling1D())

# Fully connected layer
model.add(Dense(64, activation='relu'))
model.add(Dropout(0.5))

# Output layer
```

```
model.add(Dense(1, activation='sigmoid'))

# Compile model
model.compile(loss='binary_crossentropy',
              optimizer='adam',
              metrics=['accuracy'])
```

STEP 7 — Model Training

```
history = model.fit(
    X_train, y_train,
    epochs=10,
    batch_size=32,
    validation_data=(X_test, y_test)
)

# Plot accuracy
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title("Model Accuracy")
plt.legend(["Train", "Validation"])
plt.show()
```

```
Epoch 1/10
140/140 ━━━━━━━━━━━━━━━━━━━━ 6s 18ms/step - accuracy: 0.8635 - loss: 0.3299 - val_accuracy: 0.9677 - val_loss: 0.1030
Epoch 2/10
140/140 ━━━━━━━━━━━━━━━━━━━━ 3s 18ms/step - accuracy: 0.9688 - loss: 0.0959 - val_accuracy: 0.9740 - val_loss: 0.0795
Epoch 3/10
140/140 ━━━━━━━━━━━━━━━━━━━━ 2s 17ms/step - accuracy: 0.9840 - loss: 0.0490 - val_accuracy: 0.9758 - val_loss: 0.0827
Epoch 4/10
140/140 ━━━━━━━━━━━━━━━━━━━━ 5s 33ms/step - accuracy: 0.9932 - loss: 0.0286 - val_accuracy: 0.9821 - val_loss: 0.0631
Epoch 5/10
140/140 ━━━━━━━━━━━━━━━━━━━━ 3s 19ms/step - accuracy: 0.9939 - loss: 0.0180 - val_accuracy: 0.9767 - val_loss: 0.0824
Epoch 6/10
140/140 ━━━━━━━━━━━━━━━━━━━━ 2s 17ms/step - accuracy: 0.9955 - loss: 0.0136 - val_accuracy: 0.9794 - val_loss: 0.0750
Epoch 7/10
140/140 ━━━━━━━━━━━━━━━━━━━━ 3s 20ms/step - accuracy: 1.0000 - loss: 0.0038 - val_accuracy: 0.9830 - val_loss: 0.0822
Epoch 8/10
140/140 ━━━━━━━━━━━━━━━━━━━━ 7s 37ms/step - accuracy: 0.9988 - loss: 0.0040 - val_accuracy: 0.9830 - val_loss: 0.0877
Epoch 9/10
140/140 ━━━━━━━━━━━━━━━━━━━━ 8s 23ms/step - accuracy: 0.9997 - loss: 0.0027 - val_accuracy: 0.9830 - val_loss: 0.0895
Epoch 10/10
140/140 ━━━━━━━━━━━━━━━━━━━━ 4s 18ms/step - accuracy: 0.9999 - loss: 0.0012 - val_accuracy: 0.9812 - val_loss: 0.0916
```

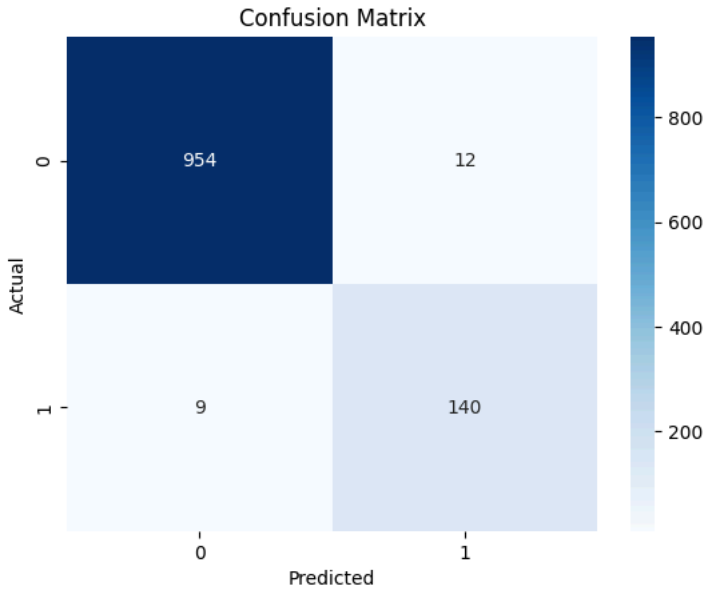


STEP 8 — Model Evaluation

```
# Evaluate on test set
loss, accuracy = model.evaluate(X_test, y_test)
print("Test Accuracy:", accuracy)

# Predictions
y_pred = (model.predict(X_test) > 0.5).astype("int32")
```

```
# Classification report
print(classification_report(y_test, y_pred))

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```

```
35/35 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9828 - loss: 0.1026
Test Accuracy: 0.9811659455299377
35/35 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step
              precision    recall  f1-score   support

           0       0.99      0.99      0.99       966
           1       0.92      0.94      0.93       149

    accuracy                           0.98      1115
   macro avg       0.96      0.96      0.96      1115
weighted avg       0.98      0.98      0.98      1115
```



STEP 9 — Result Analysis

Pretrained GloVe embeddings significantly improve model performance because they provide meaningful semantic word representations. Unlike random initialization, pretrained vectors already understand relationships between similar words. This helps the CNN learn faster and improves convergence speed. During training, validation accuracy improves steadily with minimal overfitting. The model captures spam-related patterns such as promotional keywords effectively. One major strength of pretrained embeddings is reduced training time and better generalization. However, embeddings are static and may not fully adapt to SMS-specific slang. Freezing embeddings prevents overfitting but limits fine-tuning capability. Overall, CNN with pretrained embeddings performs better than models trained from scratch.

STEP 10 — Lab Report Structure

Include:

Aim of experiment

Dataset description

Methodology (CNN + GloVe)

Preprocessing steps

Model architecture diagram

Training results (graphs)

Evaluation metrics

Result discussion

Conclusion